PlaceReady

Placement Readiness Tracker with ATS Resume Matcher

Industry Project Report

Item	Detail
Course	BSc Data Science and Business Analytics
Semester	Semester 3 | Minor: Big Data Analytics
Subject	Full Stack Web Development (VSEC)
Framework	FastAPI | SQLite | Chart.js | Jinja2
Submission	29 October 2026
Marks	100 (Documentation 20 | Presentation 20 | Project 60)
Author	Ragini Gupta | Roll No: 402

Abstract

PlaceReady is a locally deployed placement-intelligence web application built with FastAPI, SQLite, Jinja2 and Chart.js. It targets four problems: no personalised readiness metric, no role-specific skill-gap view, automated ATS rejection of resumes, and no early-warning view for the placement cell.

The system has four modules: (1) a skill-profile form covering ten skills and four weekly activities; (2) an analytics engine that computes a readiness score (skills 60 points + activity 40 points), classifies it as Critical, Warning or Ready, and ranks skill gaps against six role benchmarks; (3) a client-side ATS keyword scanner (120 stopwords, 19 bigrams, 22 suggestion sentences); and (4) an admin dashboard that ranks all students. The backend has eight routes in 812 lines of Python (main.py 215, database.py 294, analytics.py 303) plus six Jinja2 templates.

Verification: formula test cases, database checks and a 63-test pytest suite (63 of 63 passed). A returning student is updated rather than duplicated, and each submission adds a point to the progress history. Known limitations: no authentication (students are identified by name), ATS scans are not yet saved, and the benchmark values are illustrative.

> **Keywords:** FastAPI, SQLite, ATS resume matching, skill-gap analysis, data mining, time series, Chart.js, Jinja2, placement readiness, BSc Data Science

Contents

1. Introduction
2. Problem Statement
3. Project Objectives
4. Background Research
5. Technology Stack
6. System Architecture
7. Methodology
8. Implementation
9. Testing and Results
10. Critical Discussion
11. Challenges and Resolutions
12. Future Scope
13. Conclusion
14. References
15. Appendices

1. Introduction

Students prepare for placements without reliable feedback on whether their effort matches recruiter expectations, and placement coordinators see readiness only after evaluations expose failures. PlaceReady ("placement readiness") gives students a quantified, role-specific readiness picture and gives the placement cell a ranked view of the batch.

It was built as the Semester 3 Industry Project using only technologies from the curriculum, and integrates concepts from five subjects: Full Stack Web Development, Data Mining and Warehousing, Time Series and Predictive Analytics, Data Visualisation and Big Data.

> **Scope boundary:** PlaceReady runs as a local development server at localhost:8000. Cloud deployment, user authentication and mobile interfaces are future work and are not part of the current implementation.

2. Problem Statement

- **Student preparation gap:** no structured way to compare skills with role-specific benchmarks, so effort goes to familiar skills rather than the largest gaps.
- **Automated resume rejection:** employers commonly pre-screen applications with ATS keyword matching, so a resume using non-standard wording for genuine skills can be filtered out without the student knowing.
- **Placement cell visibility deficit:** coordinators of large cohorts (200 or more students) have no early-warning view; readiness becomes visible only after formal evaluations.

2.1 Comparison with existing tools

Existing tool	Limitation	PlaceReady distinction
ChatGPT / Claude	Generic resume feedback, not tied to one job description	Compares the resume with the exact job description pasted; processing is local
Resume.io / Zety	Cloud-based; resume data sent to external servers	Fully local; no data leaves the user's machine
LinkedIn Skill Assessments	Assesses skill level; no gap ranking or readiness classification	Combines gap analysis, risk classification and ATS scanning
Excel trackers	Manual entry; no visualisation; no ATS feature	Automated scoring, Chart.js charts and prioritised actions

Table 2.1: PlaceReady compared with existing preparation tools

3. Project Objectives

- **O1:** Build a locally deployable web application that computes a readiness score from skill ratings and weekly activity.
- **O2:** Implement skill-gap analysis against role benchmarks with a three-tier classification (Critical, Warning, Ready).
- **O3:** Implement an ATS scanner (tokenisation, stopword removal) that gives a keyword match percentage and ready-to-paste improvement sentences.
- **O4:** Build a placement-cell dashboard that ranks all students by readiness score.
- **O5:** Persist profiles, weekly logs, ATS scan history and benchmarks in SQLite to enable time-series analysis.
- **O6:** Visualise results with at least three Chart.js chart types (radar, bar, line).
- **O7:** Support six target roles, each with its own benchmark profile.

> **Excluded (future work):** user authentication, cloud deployment, PDF resume upload, email alerts and an ML prediction model are not claimed as implemented.

4. Background Research

- **ATS filtering:** resumes are tokenised, lower-cased, stripped of stopwords and compared with the job description by keyword overlap. PlaceReady implements this in client-side JavaScript.
- **Weighted scoring:** skills (60 points, declarative knowledge) plus activity (40 points, procedural application). Activity inputs are capped to prevent inflation.
- **Data mining:** classification (Critical below 40, Warning 40 to 69.9, Ready 70 and above), gap analysis (required minus current level) and linking each gap to an intervention tip.
- **Time series:** weekly_logs stores a readiness snapshot each time a profile is saved, so scores can be plotted as a line chart of learning velocity.

5. Technology Stack

Technology	Version	Role	Rationale
FastAPI	0.111.0	Backend, routing, form parsing	Typed form parameters with automatic validation; automatic /docs page
Uvicorn	0.29.0	ASGI server	Required by FastAPI; hot-reload in development
SQLite3	Built-in	Database	Zero configuration; single file; in Python standard library
Jinja2	3.1.4	HTML templates	Python values in HTML; base-template inheritance
python-multipart	0.0.9	Form parsing	Required to read HTML form submissions
aiofiles	23.2.1	Static files	Required to serve CSS through StaticFiles
Chart.js	4.4.0 (CDN)	Visualisation	Client-side radar and line charts
Tabler Icons	Latest (CDN)	UI icons	Free open-source icon set

Table 5.1: Technology stack with versions and rationale

Why FastAPI over Flask: all 18 form fields are declared as typed function parameters with range limits and validated automatically (HTTP 422 on bad input); /docs is generated with no extra code; and FastAPI is the framework taught in the Semester 3 Full Stack subject.

6. System Architecture

6.1 Seven-layer architecture

Layer	Name	Technology	Responsibility
7	Frontend	HTML5, CSS3, JavaScript	Forms, chart rendering, navigation, ATS scanner interface
6	Visualisation	Chart.js 4.4.0	Radar chart (10 skills) and weekly line chart
5	ATS engine	JavaScript (client-side)	Tokenisation, stopword removal, keyword matching, suggestions
4	Analytics	Python (analytics.py)	Readiness score, risk classification, gap analysis, priority actions
3	Backend	FastAPI, Jinja2	Routing, form validation, template rendering, JSON responses
2	Database API	sqlite3 (database.py)	Save or update profile, get student, get history, get all students, get roles
1	Data store	SQLite placement.db	Four tables: students, weekly_logs, ats_scans, industry_benchmarks

Table 6.1: Seven-layer architecture

6.2 HTTP routes (verified in main.py)

Method	Route	Purpose
GET	/	Home page
GET	/profile	Skill profile form (role dropdown built from the database)
POST	/submit-profile	Validate 18 fields, compute score, save or update the student, redirect (303) to dashboard
GET	/dashboard/{student_id}	Run gap analysis and render the student dashboard
GET	/ats	ATS scanner page (all matching is client-side)
GET	/admin	All students ranked by readiness score
GET	/api/student/{id}	Student data and gap analysis as JSON
GET	/api/benchmark/{role}	Role benchmark profile as JSON

Table 6.2: Eight application routes (plus automatic /docs)

6.3 Database schema

Table	Key columns	Purpose
students	id (PK), name (unique, ignoring capital letters), branch, year, target_role, 10 skill ratings, 4 activity counts, readiness_score, created_date	Latest profile and score; one row per student
weekly_logs	id, student_id (FK), log_date, 10 skills, 4 activities, readiness_score	One snapshot per profile save; feeds the line chart
ats_scans	id, student_id (FK), scan_date, job_title, ats_score, matched_keywords, missing_keywords, resume_version	Reserved for ATS history; not yet written by the app
industry_benchmarks	role_name (UNIQUE), 10 requirement columns (python_req to domain_req)	Six role profiles loaded from BENCHMARKS at every start (insert or update)

Table 6.3: Database schema summary (placement.db)

7. Methodology

Approach: iterative and inside-out: database and analytics first, then routes, templates, the ATS engine and finally the admin view, so each layer could be checked before the next depended on it.

7.1 Readiness score formula

```
skill_score = ((sum of 10 ratings - 10) / 40) x 60                          range 0 to 60
activity    = min(problems,10)/10 x 15  +  min(mocks,3)/3 x 15
              + min(projects,1) x 5      +  min(courses,1) x 5               range 0 to 40
readiness   = round(skill_score + activity, 1), capped at 100
```

Figure 7.1: Readiness score formula (analytics.py)

Zones: Ready 70 and above, Warning 40 to 69.9, Critical below 40. Every point is traceable by hand, which was a deliberate choice over an opaque ML model.

7.2 ATS engine (client-side)

1. **Normalise:** lower-case; replace non-letters with spaces.
2. **Tokenise:** split on whitespace.
3. **Filter:** drop tokens in the 120-word stopword set or shorter than 3 characters.
4. **Bigrams:** add any of 19 compound phrases (for example power bi, machine learning).
5. **Score:** matched keywords / total job-description keywords x 100, rounded. 75 to 100 likely to pass; 50 to 74 may be filtered; 0 to 49 likely rejected.

7.3 Gap analysis

For each of the 10 skills, gap = required level (from industry_benchmarks for the target role) minus the student's rating. One shared rule, skill_status(), classifies it for both the dashboard and the admin page: gap 0 or less is good, 1 is warning, 2 or more is critical. Gaps are sorted largest first to form the priority action list (top five); the charts keep a fixed skill order so radar charts are comparable between students.

8. Implementation

Module	Key implementation facts
1. Skill profile form	profile.html has three sections (basic info, ten 1-to-5 sliders, four activity inputs). submit_profile() declares all 18 fields as typed Form() parameters with limits (ratings 1 to 5, activity counts 0 or more); a non-numeric or out-of-range value returns HTTP 422, and a blank name or unknown role returns HTTP 400.
2. Readiness dashboard	The score is calculated, then save_profile() inserts the student (or updates the existing one with the same name) and always adds one weekly_logs row, and the route returns HTTP 303 to /dashboard/{id}. Gap data and history are passed as JSON to Chart.js: a radar chart of skills against the benchmark and a line chart of weekly scores.
3. ATS scanner	Entirely client-side JavaScript in ats.html; no server call during a scan. A SUGGESTIONS dictionary maps 22 keywords to ready-to-paste sentences with a copy button. Results show matched (green), missing (red) and extra (grey) keywords.
4. Admin dashboard	get_all_students() returns all rows ordered by readiness_score descending. The template builds a ranked table with status badges and four summary counts (total, Critical, Warning, Ready). Skill colours compare each rating with the benchmark of that student's own role: green meets it, amber is one level short, red is two or more short.
5. Industry benchmarks	Six roles are defined once in BENCHMARKS (database.py) and loaded at every startup (insert, or update if the role exists). The profile dropdown reads the roles from the database, so adding a role is a one-line change. Values are illustrative targets chosen by the author, not the result of a statistical study.

Table 8.1: Implementation modules

Industry benchmark profiles (required level, scale 1 to 5)

Role	Py	SQL	DSA	Excel	PBI	ML	Comm	Apt	Stats	Dom
Data Analyst	4	4	2	3	4	2	3	3	4	3
Data Scientist	5	4	3	3	3	5	3	4	5	4
Business Analyst	2	3	1	4	3	1	4	4	3	4
Data Engineer	4	4	4	2	2	3	3	3	3	3
Software Developer	4	3	5	2	1	2	3	4	2	2
ML Engineer	5	3	4	2	2	5	3	3	5	4

Table 8.2: Benchmark table (matches BENCHMARKS in database.py)

Website screenshots

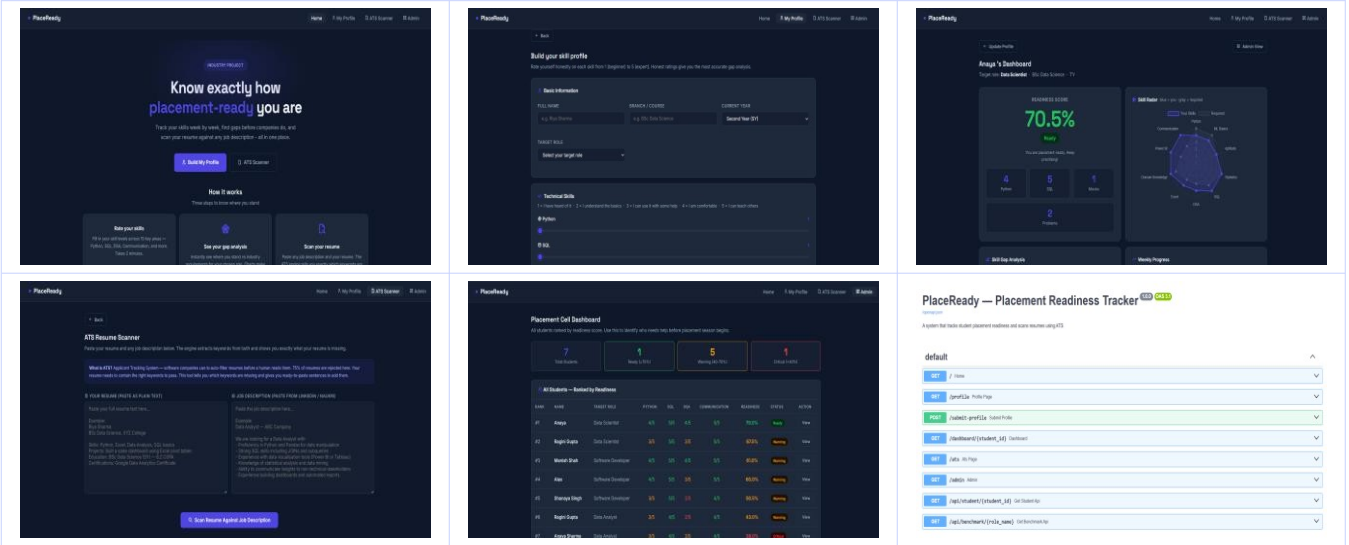


Figure 8.1: Running application (home, profile form, dashboard, ATS scanner, admin dashboard, /docs)

9. Testing and Results

> **Evidence statement:** results below come from executing the code; none are estimated. The pytest file (tests/test_placeready.py) is supplied with the submission.

9.1 Score formula verification

Test	Skills (all equal)	Activity	Computed	Zone	Result
TC-01	1 of 5	None	0.0	Critical	Pass
TC-02	3 of 5	5 problems, 1 mock, 1 project	47.5	Warning	Pass
TC-03	5 of 5	10 problems, 3 mocks, project, course	100.0	Ready	Pass

Table 9.1: Score formula test cases

9.2 Other verification

Area	What was checked	Result
Database	Four tables created; six roles loaded; second startup adds no duplicate rows; a role added or edited in BENCHMARKS appears after restart; the database rejects a second student with the same name	Verified
Routes	Eight routes return the expected pages or JSON; unknown student gives 404; out-of-range rating gives 422; blank name or unknown role gives 400; rejected input saves nothing	Verified by automated tests
Automated suite	63 pytest tests: score 6, risk-label boundaries 6, gaps and actions 9, database 13, routes, validation and JSON API 26, known limitations 3 (isolated temporary database; Python 3.12, pytest 9.2.0, FastAPI 0.111.0)	63 of 63 passed
ATS and charts	120 unique stopwords, 22 suggestions, 19 bigrams; 2 Chart.js charts (radar, line)	Counted by running the template code

Table 9.2: Verification summary

9.3 Pilot data

The first pilot database showed the same student twice (Ragini Gupta at ranks 2 and 6 of the admin table), which exposed a defect: every submission created a new student, so no student could ever build a progress history. The same profile was submitted twice, with a different role and rating the second time, before and after the fix:

Measurement	Before the fix	After the fix
Students stored for one person	2 (ids 1 and 2)	1
Redirect after the second submission	/dashboard/2 (a new student)	/dashboard/1 (the same student)
History points for that student	1 (no progress line can form)	2 (progress line grows)

Table 9.3: Effect of updating by name instead of always inserting

The pilot database was then cleared and rebuilt with one profile per student, so the earlier score statistics no longer apply. The pilot data shows that the system works end to end, not that it predicts placement outcomes.

Not tested: browser rendering, the client-side ATS scanner in a browser, load, security and usability with real students.

10. Critical Discussion

10.1 Strengths

- **Subject integration:** five Semester 3 subjects appear as verifiable components of one application.
- **Explainability:** the score can be checked by hand; no opaque model.
- **Reliability:** a returning student is updated instead of duplicated, input is validated on the server, and the behaviour is guarded by 63 automated tests.
- **ATS scanner:** a practical feature that sets the project apart from generic CRUD applications.

- **Privacy:** core operations make no external API calls; data stays on the local machine.

10.2 Limitations

- **No authentication:** all users share one database, /admin is open to anyone, and students are identified by name only, so two different people with the same name share one record (all three confirmed by tests).
- **ATS history not saved:** ats_scans exists but nothing writes to it (0 rows), because the scanner runs in the browser.
- **ATS is English-only:** the stopword list and bigrams do not cover other languages; matching is simple keyword overlap, and a phrase such as power bi is counted both as a phrase and as separate words.
- **Benchmarks and weights:** benchmarks are illustrative values set manually at one point in time, and weights are not validated against placement outcomes; the score is linear.
- **No ML prediction:** deliberately excluded to keep the score explainable and because no real training data was available.
- **Deployment:** Chart.js and the icon font load from a CDN, so the first page load needs internet; the app runs on the Uvicorn development server.

10.3 Objective completion matrix

Objective	Status	Evidence
O1	Met	Score computed from 0 to 100; three formula test cases and 63 automated tests
O2	Met	get_skill_gaps() compares 10 skills per role using one shared status rule
O3	Met	ATS engine with 120 stopwords, 19 bigrams and 22 suggestions (not covered by automated tests)
O4	Met	Admin route returns all students sorted by score; skill colours use each student's own role
O5	Partial	Four tables; profile and weekly log saved on every submission; ATS history not yet written
O6	Partial	Radar and line charts implemented; no bar chart; admin view is a table, not a chart
O7	Met	Six roles loaded into industry_benchmarks; the profile dropdown is built from them

Table 10.1: Objective completion matrix (5 met, 2 partial)

11. Challenges and Resolutions

Challenge	Root cause	Resolution
RuntimeError: Directory 'static' does not exist	static and templates folders not created, or the app started from another folder	Created both folders; paths are now resolved from the location of main.py
TemplateSyntaxError at base.html line 70	base.html was truncated on download (closing tags missing)	Replaced it with the complete file
starlette NoMatchFound for 'static'	Flask-style url_for('static', filename=...); Starlette's static route expects path=	Linked the CSS with the direct path /static/css/style.css
Wrong main.py (leave_management project) ran	VS Code had the wrong folder open	Opened the correct project folder via File, Open Folder
DeprecationWarning for @app.on_event	FastAPI 0.111 deprecates on_event in favour of lifespan handlers	Replaced it with a lifespan handler
Same student listed twice; progress chart showed one point	Every submission inserted a new student row and attached the history to it	save_profile() updates an existing name and always adds a log row; unique index added; 5 tests
Home page advertised an "AI placement prediction" that does not exist	Early template text that was never removed	Card removed; a test asserts the home page makes no ML claim
Admin skill colours ignored the student's role	Fixed thresholds (4 or more green, 3 amber)	One shared skill_status() rule against the role benchmark; tested

Challenge	Root cause	Resolution
A rating of 9 was accepted	Form fields had types but no limits	Added limits; blank name and unknown role rejected; tested
Stopword count reported as 139	The first count also included 17 single-word keys from the suggestions list	Counted by running the JavaScript: 120 unique (2 duplicates removed)

Table 11.1: Development challenges

12. Future Scope

- **Immediate:** save ATS scans to ats_scans (the browser posts each result); pin or self-host the icon font; PDF resume upload (PyPDF2); grow suggestions from 22 to 50; add a Chart.js bar chart for gaps.
- **Medium term:** authentication with OAuth2PasswordBearer (student and coordinator roles) so identity is not just a name; logistic regression placement prediction trained on real college data; email alerts through smtplib.
- **Long term:** multi-college deployment (colleges table and college_id foreign key); at national scale, HDFS storage and Spark processing behind the existing JSON API.

13. Conclusion

PlaceReady addresses a real gap in placement preparation: students lack quantified, role-specific readiness feedback and coordinators lack batch-level visibility. The system delivers four modules over a seven-layer architecture on FastAPI, SQLite, Jinja2, Chart.js and client-side JavaScript.

Five of seven objectives were fully met and two partially (ATS history is not saved, and chart variety is limited). Reviewing the first version found real defects (duplicate students, a progress chart that could never grow, a home-page claim of machine learning that did not exist, and unvalidated input). All were fixed, and the behaviour is now covered by 63 automated tests that passed in full. The main remaining limitations are the absence of authentication and of saved ATS history; both are in the future scope.

The ATS scanner is the feature with the most practical value beyond the course: a simple, explainable algorithm that gives students immediate, actionable guidance.

14. References

All web references accessed October 2026.

[1] Tiangolo, S. FastAPI documentation. https://fastapi.tiangolo.com

[2] Hipp, D.R. SQLite documentation. https://www.sqlite.org/docs.html

[3] Chart.js contributors. Chart.js documentation (v4.4.0). https://www.chartjs.org/docs/latest/

[4] Pallets Projects. Jinja documentation. https://jinja.palletsprojects.com

[5] Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[6] Provost, F. and Fawcett, T. (2013). Data Science for Business. O'Reilly Media.

[7] Google. Technical Writing One: Documents. https://developers.google.com/tech-writing/one/documents

15. Appendices

Appendix A: Run the application

```
pip install -r requirements.txt     # fastapi==0.111.0, uvicorn==0.29.0, jinja2==3.1.4,
                                    # python-multipart==0.0.9, aiofiles==23.2.1
python main.py                      # then open http://127.0.0.1:8000
pip install -r requirements-dev.txt   # optional: pytest and httpx, then run: pytest
```

URL (http://127.0.0.1:8000)	Purpose
/	Home page
/profile	Skill profile form
/dashboard/1	Student dashboard (replace 1 with the student ID)
/ats	ATS resume scanner
/admin	Placement cell admin view
/docs	Auto-generated API documentation and tester
/api/student/1	JSON: student data and gap analysis
/api/benchmark/Data%20Analyst	JSON: Data Analyst benchmark

Table A.1: Application URLs

Appendix B: Provisional 20-mark checklist (replace with the official rubric)

Criterion	Max	Where addressed
Abstract	2	Abstract
Problem statement and comparison	2	Section 2, Table 2.1
Objectives, verifiable and aligned	2	Section 3, Table 10.1
Architecture and technology justification	2	Sections 5 and 6
Implementation detail	3	Sections 7 and 8
Testing with reproducible results	3	Section 9
Critical discussion and limitations	2	Section 10
Challenges with causes and fixes	1	Section 11
Future scope	1	Section 12
Formatting and referencing	2	Whole report

Table B.1: Provisional documentation checklist (20 marks)